

CGI Capacity Analyzer for Director Workload and RFP Assignment

Freya Kan, Jai, Junxian Lin, Qian Yang, Yixiao Jing

Master of Data Science in Computational Linguistics

University of British Columbia

Abstract

CGI Atlantic / Media Atlantic needs a more consistent way to understand director workload and assign incoming Requests for Proposals (RFPs). We developed the CGI Capacity Analyzer, a transparent decision-support prototype that combines anonymized CRM opportunity data with historical proposal and response text. The prototype includes two stakeholder-facing deliverables: a Director Capacity Dashboard that visualizes director workload relative to each director’s historical norm, and an RFP Assignment Tool that uses capacity signals, retrieved supporting examples, service-fit evidence, and risk flags to suggest suitable directors for review.

Because the project does not include labeled ground truth for director capacity, RFP effort, or correct assignment, we evaluated the prototype through implementation tests, preprocessing regression checks, retrieval validation, browser-style QA, and qualitative evidence review rather than standard supervised-learning metrics. The final prototype supports local data replacement for CGI testing, preserves RFP alias fields for CRM linkage review, and maintains fallback-safe behavior when retrieval or enrichment components are unavailable. Our main finding is that the system is useful as an inspectable starting point for workload review and RFP assignment, but its recommendations still require CGI validation before they can support real staffing decisions.

1 Introduction

CGI Atlantic / Media Atlantic needs a more consistent, data-informed way to understand how work is distributed across directors and how new Requests for Proposals (RFPs) should be assigned. When capacity information is tracked manually or inconsistently, stakeholders may struggle to see which directors are available, which directors are near their normal workload, and which directors are already overextended. This is a practical business

problem rather than only a technical one: an assignment suggestion is useful only if stakeholders can inspect the evidence behind it and decide whether it makes sense in context.

Figure 1 illustrates this bottleneck: new RFP work continues to arrive while directors may already carry very different project loads. The goal of this project is to make those workload differences more visible and to support more consistent assignment decisions.

To address this problem, we developed the CGI Capacity Analyzer, a transparent decision-support prototype for workload review and RFP assignment. The prototype is designed to organize signals that CGI stakeholders can inspect, rather than to make staffing decisions automatically. This framing is important because the project does not include manually labeled ground truth for director capacity, RFP effort, or correct director assignment.

The system has two stakeholder-facing deliverables. The first is a Director Capacity Dashboard, which summarizes each director’s current workload, historical norm, relative workload, capacity label, and workload drivers. The dashboard uses relative workload as the main business-facing signal because it compares a director’s current workload with that same director’s historical pattern, rather than treating all directors as if they had the same baseline. The second deliverable is an RFP Assignment Tool. The tool compares submitted RFP text with historical proposal and response material, then combines retrieved supporting examples with capacity signals, service-fit evidence, and risk flags to suggest suitable directors for stakeholder review.

The main contributions of this project are:

- a reproducible, opportunity-level data workflow that combines two anonymized CRM opportunity exports while avoiding duplicate-opportunity expansion;

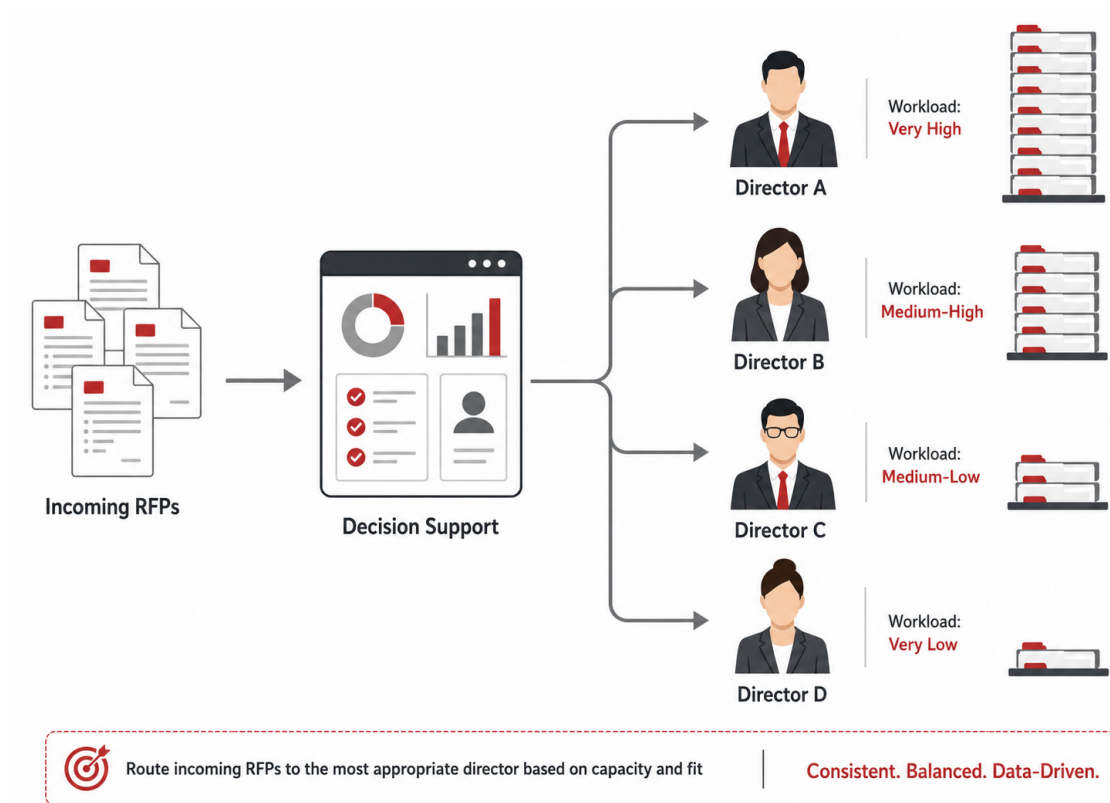


Figure 1: Conceptual illustration of the project motivation. Incoming Requests for Proposals (RFPs) need to be assigned while directors may already carry different workload levels, motivating a decision-support workflow that considers both capacity and fit. The illustration was generated using ChatGPT as a visual aid.

- a Director Capacity Dashboard that makes workload distribution inspectable through current workload, historical norm, relative workload, workload drivers, and stakeholder-readable capacity labels;
- an RFP Assignment Tool that combines proposal-text evidence, capacity information, service-fit signals, and risk flags to support director assignment review;
- a fallback-safe and locally updatable handoff workflow that allows CGI to regenerate processed outputs from local data sources while keeping sensitive files outside version control; and
- an evaluation framing that distinguishes implementation confidence from business validation in the absence of ground-truth assignment labels.

The rest of this report is organized as follows. Section 2 describes the data sources, preparation steps, and known limitations. Section 3 explains the system design, capacity scoring, retrieval

pipeline, recommendation logic, enrichment behavior, and handoff-oriented data replacement workflow. Section 4 describes the experimental pipeline used to validate the prototype without ground-truth labels. Section 5 presents the main results and analysis. Section 6 prioritizes future work, and Section 7 summarizes the main takeaways. Appendix A preserves the project timeline and team contributions.

2 Data

2.1 Data Sources

The project uses two anonymized Excel CRM opportunity exports and one JSON file containing historical proposal and response text. The two opportunity exports are referred to as opps1 and opps2. The proposal data contains 19 top-level historical proposal records, from which the preprocessing workflow extracts proposal and response text sections when available. These CRM exports are not static: CGI updated them during the project, so the preprocessing pipeline is built to regenerate cleaned outputs from replaced spreadsheets. One such update added a machine-readable RFP alias

column (observed as `Json S-Num`) to `opps1`, raising its meaningful column count from 20 to 21; the workbook may contain additional blank or formatted columns, but these are not treated as data fields. Table 1 summarizes the three data sources: the two Excel exports carry the structured opportunity fields used for capacity scoring, while the JSON supplies the proposal and response text used by the RFP pipeline.

All CGI-provided files are treated as sensitive local data. They are not committed to GitHub. Local credentials and API configuration are also excluded from version control. Although the project was designed around the provided documents, the preprocessing workflow lets CGI update the data source locally: it can point the pipeline to replaced opportunity exports, regenerate processed outputs, and test the app without committing raw spreadsheets or generated outputs to the repository.

2.2 Opportunity Data

Initial EDA indicates that `opps1` and `opps2` are different views of largely overlapping CRM opportunity data. We use `opps2` as the primary base table because it contains the richer CRM schema and has one unique `opportunity_id` per row in the current export. We use `opps1` as a secondary source: selected fields from `opps1` are collapsed to one row per `opportunity_id` and merged into `opps2`, while `opps1`-exclusive opportunities are appended to the final `opportunity_df`. Fields that appear under different names across the two exports, especially revenue-related fields, require normalization and validation before downstream scoring and are handled through the validation and fallback process.

Because the capacity model operates at the opportunity level, this distinction matters: `opps1` contains duplicate `opportunity_id` values that reflect service- or product-level detail rather than distinct opportunities, so a naive merge would double-count workload. The cleaned table therefore preserves one row per opportunity; the merge logic that enforces this is described in Section 3, and the full audit is reported in Appendix A. Table 2 summarizes the result: 8,746 rows (8,695 `opps2` base rows plus 51 `opps1`-exclusive opportunities) with zero duplicate IDs.

The updated `opps1` spreadsheet also introduced the `Json S-Num` field, which is normalized and preserved in the cleaned opportunity output as `rfp_alias`, with `has_rfp_alias` indicat-

ing whether a row contains an alias. Excel cell highlighting, including orange-highlighted rows, is treated as a manual visual annotation and is not used as pipeline logic.

2.3 Proposal and RFP Text Data

The proposal JSON contains 19 historical proposal records together with their associated responses; the two document types are not paired one-to-one. Extracting them yields 46 text documents from the 19 top-level records, made up of 19 proposal documents and 27 response documents. These documents are text-heavy and many are too long to use as single retrieval inputs, so they are later split into smaller chunks for the RFP pipeline.

Each chunk carries retrieval metadata (identifiers, source type, and the text itself; the chunking procedure is detailed in Section 3). Critically, the original proposal JSON does not by itself populate the linkage fields needed to tie a chunk to a director or CRM opportunity. Without deterministic alias linkage, proposal text alone cannot prove director involvement or opportunity ownership—a limitation that shapes the methods and evaluation that follow.

The preserved `rfp_alias` field improves traceability when a proposal or RFP title contains an S-number that deterministically matches a cleaned opportunity row. When such a match exists, retrieved chunks can carry linked CRM context for stakeholder review. This linked context is useful metadata, but it should be interpreted as contextual support rather than proof of proposal authorship or final assignment suitability.

2.4 Data Preparation and Known Limitations

The data preparation workflow is designed around a few conservative constraints. First, the final opportunity table must preserve opportunity-level grain, so duplicate `opportunity_id` rows from `opps1` are collapsed rather than used to expand downstream records. Second, proposal text is chunked because many proposal and response documents are too long for single-input retrieval. Third, all raw CGI-provided files, generated processed outputs, and local credentials remain outside version control.

The project does not contain labeled capacity categories, measured director workload hours, labeled RFP effort, or gold-standard director assignments. Proposal chunks also cannot always be reliably linked to directors or opportunities. Even when

Data source	Approx. size	Main contents	Role in project
opps1 CRM export	8,435 rows 20 original columns; 21 after the alias column was added	Supplemental product, service-solution, IP, service-solution revenue, opportunity-level fallback revenue, delivery territory fields, and, in the updated export, a machine-readable RFP alias field observed as Json S-Num.	Secondary supplemental source. Selected fields are collapsed to one row per opportunity_id before merging into opps2. The alias field is preserved as rfp_alias / has_rfp_alias when present to support RFP-to-CRM linkage review.
opps2 CRM export	8,695 rows 33 columns	Richer CRM opportunity schema with owner, status, sales stage, probability, revenue, date, duration, and proposal-related fields.	Primary base table for opportunity_df. It defines the opportunity-level grain for capacity scoring and dashboard inputs.
Proposal / response JSON	19 top-level records; 46 extracted text documents	Historical proposal and response text.	Used for RFP text preprocessing, historical corpus chunking, retrieved supporting examples, assignment-context generation, Azure/Chroma backend retrieval validation, fallback-safe local retrieval, and alias-based linkage review when a matching rfp_alias is available.

Table 1: Overview of the main project data sources. The files are stored locally and are not committed to GitHub because they contain CGI-provided data.

Merge statistic	Value
opps1 rows	8,435
opps2 rows	8,695
opps1 unique IDs	7,679
opps2 unique IDs	8,695
Matched opportunity IDs	7,628
opps1-exclusive IDs	51
opps2-only IDs	1,067
Final opportunity_df rows	8,746
Duplicate IDs in final output	0

Table 2: Opportunity merge audit summary. The final output preserves opportunity-level grain by avoiding one-to-many expansion from duplicated opps1 records.

alias-based linkage is available, linked CRM meta-data provides contextual support for stakeholder review rather than proof of proposal authorship, prior director involvement, delivery responsibility, staffing ownership, or final assignment suitability. These data limitations directly shape the methods and experimental pipeline described in the next sections.

3 Methods

3.1 System Architecture and Deliverables

The project has two stakeholder-facing deliverables. The first is the Director Capacity Dashboard, which translates cleaned CRM opportunity data

into director-level workload signals. It displays each director’s current workload, historical baseline, relative_load, capacity label, and workload drivers such as open opportunities, late-stage opportunities, weighted pipeline, and inferred delivery commitments. The dashboard is designed to make the capacity signal inspectable: stakeholders can see not only whether a director is labelled Available, Near Historical Norm, High Load, Overextended, or No baseline, but also which workload components contribute to that label.

The second deliverable is the RFP Assignment Tool. For a submitted RFP, the tool estimates effort, retrieves supporting proposal or response examples from the historical or fallback corpus, and recommends directors using available capacity signals, retrieved evidence, service-domain overlap, and risk flags. When real or pre-built capacity_df is available, the Streamlit RFP page passes it into generate_assignment_context(), so the recommendation cards can use capacity-aware signals rather than relying only on mock director data. When deterministic rfp_alias linkage is available, retrieved examples can also show linked CRM context for stakeholder review.

The architecture remains modular. The opportunity-cleaning pipeline produces

opportunity_df and downstream cleaned datasets for validation and scoring. The capacity engine computes director-level workload, relative_load, capacity_score, and capacity labels. The Streamlit app displays the dashboard and provides the RFP input and output interface. The RFP pipeline preprocesses proposal and response text, creates retrieval chunks, uses Azure/Chroma retrieval when the preferred backend path is available, preserves deterministic local fallback retrieval, propagates linked CRM metadata when alias-based matches exist, and returns a stable assignment_context schema for display.

The interactive application is implemented in Streamlit, while the preferred retrieval path uses Azure OpenAI embeddings with Chroma vector-store retrieval when configuration and dependencies are available (Snowflake Inc., 2026; Microsoft, 2026a,b; Chroma, 2026).

Figures 2 and 3 show the two main pages of the final prototype. Figure 2 illustrates why relative_load is foregrounded: the dashboard compares each director's current workload with that director's own historical norm, while also showing label mix, workload trend, and director-level summary fields. Figure 3 shows how the RFP page presents the active retrieval mode, prototype caveat, submitted RFP input, Retrieved Supporting Examples, capacity-aware director recommendations, and risk flags in one reviewable workflow.

3.2 Capacity Scoring

The current capacity model is transparent and formula-based. Current workload is represented as the sum of sales load and inferred delivery load. Sales load uses open opportunities, stage weight, probability, authoritative revenue, and project duration. Delivery load uses won active deliveries and monthly revenue as a proxy for delivery commitment. This approach keeps the scoring logic interpretable and avoids using an LLM as the source of truth for numeric capacity scores.

In the handoff-ready prototype, relative_load remains the primary business-facing capacity metric. It is defined as a director's current workload divided by that same director's historical average workload. This interpretation is useful for communicating overextension because it can distinguish a director who is slightly above normal workload from one who is carrying multiple times their historical norm. capacity_score remains in the back-

end output for compatibility and for downstream RFP assignment scoring, but it is no longer treated as the main dashboard metric because it is capped once relative_load reaches or exceeds the baseline.

Capacity labels are assigned from relative_load. The current prototype labels directors as Available, Near Historical Norm, High Load, Overextended, or No baseline. These thresholds are project-side calibration choices based on the observed director-level distribution and the business interpretation that carrying at least twice the historical workload should be considered overextended. CGI should review and adjust these thresholds before production use.

The dashboard filtering logic is designed to keep the historical baseline stable. Territory and opportunity-owner filters define the selected director population for both current workload and historical baseline. Row-level filters such as status, sales stage, opportunity type, sales model, and created-date range narrow the current workload and trend views, but do not recompute the historical baseline denominator. This allows filtered views to answer how a selected slice of current workload compares with the selected directors' normal workload.

3.3 RFP Retrieval Pipeline

In terms of the RFP assignment tool, we combined proposal text preprocessing, retrieval-oriented evidence, structured capacity signals, service_solution fit, heuristic scoring, optional LLM enrichment, and alias-based CRM context in the current prototype when available. The RFP pipeline extracts proposal or response text from proposals_responses.json when usable, creates chunks for the local proposal corpus, and uses a built-in sample corpus only when the local proposal corpus is not available. The tool accepts new RFP input as pasted text or as uploaded TXT, DOCX, or text-based PDF files. The submitted RFP text serves as query text against the chunked historical or sample corpus, and the dashboard-facing assignment_context is produced from retrieved supporting examples, capacity data, service-fit signal, risk flags, possible LLM enrichment, and linked CRM metadata whenever deterministic rfp_alias linkage is available.

To prepare the historical retrieval corpus, the pipeline extracts text from proposals_responses.json and splits it into manageable segments to respect embedding token

limits and improve retrieval precision. Each generated chunk is systematically enhanced with structured metadata to ensure traceability. To be more specific, the required metadata fields include `proposal_id`, `chunk_id`, `document_id`, `source_type`, `chunk_index`, and the extracted text itself. The schema also preserves `opportunity_owner` and `opportunity_id` fields to support potential CRM linkage, although they often remain null in the raw, unlinked historical dataset.

The preferred retrieval architecture uses Azure OpenAI embeddings with Chroma vector-store retrieval. After CGI provided the remaining Azure OpenAI configuration values, Azure embedding access was validated through backend smoke tests, sample retrieval validation, and local proposal-corpus retrieval checks. When Azure configuration is available and the preferred path succeeds, the RFP engine uses Azure-generated embeddings and Chroma retrieval. If Azure embedding is unavailable, Chroma storage is unavailable, a Chroma query fails, or the required dependencies are not installed, the system falls back to deterministic local retrieval while preserving the same dashboard-facing `assignment_context` schema. The Streamlit RFP page reads the retrieval mode from the backend output, so users can distinguish Azure/Chroma retrieval from local fallback retrieval.

3.4 Recommendation Logic and Output Schema

The RFP recommendation logic combines retrieval evidence with structured director capacity signals. In the final integration stage, the Streamlit RFP page calls `generate_assignment_context(rfp_text, director_df=capacity_df)` when capacity data is available. This allows recommended directors to use real or pre-built values such as `capacity_label`, `capacity_score`, and `relative_load`. If only synthetic or schema-aligned mock capacity data is available, the page labels the capacity data source so the output is understood as fallback prototype guidance.

The dashboard-facing `assignment_context` schema includes an RFP summary, an effort object displayed as Estimated Effort, Retrieved Supporting Examples, recommended directors, risk flags, and notes. Recommended-director cards can display capacity label, capacity score, relative load, assignment score, match reason, ca-

capacity explanation, experience-match explanation, and supporting chunks. In the final prototype, retrieved examples can also carry linked CRM context such as `rfp_alias`, `opportunity_id`, `opportunity_owner`, `service_solution`, status fields, and opportunity outcome when alias-based linkage is available.

The assignment score is designed as a decision-support signal rather than a final ranking of staffing suitability. In the current prototype, director recommendations are capacity-first and evidence-informed. The scoring logic combines several inputs: the director's capacity score and relative load, the strength of retrieved supporting examples, simple service-domain overlap between the submitted RFP and available CRM service fields, and linked CRM context when a deterministic `rfp_alias` match is available. Capacity receives the largest influence because the tool is intended to help CGI avoid assigning new work to already overloaded directors. Retrieval and service-fit signals then provide supporting evidence about the type of work described in the RFP.

When linked CRM metadata is available, the tool can show opportunity context such as `opportunity_id`, `opportunity_owner`, `opportunity_manager`, `service_solution`, status, and opportunity outcome. This improves reviewability because stakeholders can see which CRM opportunity a retrieved proposal example appears to relate to. However, this linkage is treated conservatively. It provides CRM context for the retrieved example, but it does not prove proposal authorship, delivery responsibility, or that the listed owner should automatically receive a similar future assignment.

The recommendation output therefore presents each director with both a numeric assignment score and explanatory text. The numeric score helps order the shortlist, while the explanation clarifies the underlying signals: capacity label, relative load, retrieved examples, service-fit overlap, and any relevant caveats. This allows the RFP page to support human review rather than replace it.

The UI wording is intentionally conservative. Retrieval results are labelled Retrieved Supporting Examples rather than Similar Historical RFPs because the chunks provide semantic support, not confirmed assignment history. Scores shown beside retrieved examples are described as match scores. Director cards are framed as prototype-fit recommendations based on capacity signals, service-fit

signals, and retrieved evidence. This wording helps stakeholders inspect the recommendation without reading it as a final staffing decision.

3.5 LLM Enrichment and Fallback Behavior

Azure OpenAI chat enrichment is used only for explanatory text. When the chat deployment is configured and the response is usable, `generate_assignment_context()` can enrich the RFP summary, effort explanation, and per-director match reasons. Numeric capacity signals and retrieval status do not come from the LLM. If chat enrichment is unavailable, fails, or returns malformed JSON, the system falls back to deterministic heuristic output while preserving the same display schema.

The RFP page separates retrieval mode from LLM enrichment status. Retrieval mode tells stakeholders how supporting examples were found, such as Azure/Chroma retrieval or local fallback retrieval. LLM enrichment status tells stakeholders whether generated explanatory text was added. Keeping these statuses separate reduces the risk that users confuse a local retrieval fallback with an Azure OpenAI chat fallback, or interpret an LLM-written explanation as validated business analysis.

The page also displays the capacity data source when available, so stakeholders can distinguish recommendations using computed CRM-derived capacity data from recommendations using fallback or sample capacity inputs. Together, retrieval mode, LLM enrichment status, capacity data source, match-score wording, and risk flags make the RFP output more readable while preserving the report's decision-support framing.

3.6 Client Data Replacement and Alias-Based Linkage

For handoff, the preprocessing workflow was updated so CGI can run preprocessing with local spreadsheet paths through command-line arguments, including configurable `opps1` and `opps2` paths, sheet name, and processed-output directory. The original no-argument workflow was preserved, so the cleaner still defaults to the `demo/anonymized` filenames when no custom paths are supplied. This design allows CGI to place local spreadsheets under the local data directory, regenerate processed CRM-derived outputs under `data/processed`, and run the Streamlit app without committing raw spreadsheets or generated processed data to GitHub.

The updated `opps1` file was inspected and the new `Json S-Num` field was preserved through preprocessing as `rfp_alias`, with `has_rfp_alias` marking rows where the alias is present. Building on the preserved alias field, the RFP pipeline can map proposal or RFP title S-numbers to cleaned opportunity rows when a deterministic alias match is available. Linked metadata such as `rfp_alias`, `opportunity_id`, `opportunity_owner`, `opportunity_manager`, `service_solution`, `status`, `status_reason`, and `opportunity_outcome` can be attached to proposal chunks and propagated through retrieval outputs. This improves traceability by allowing stakeholders to inspect retrieved semantic evidence together with related CRM context.

However, linked CRM metadata is treated as contextual support for review, not as proof of proposal authorship, director experience, delivery responsibility, staffing ownership, or final assignment suitability. Orange-highlighted spreadsheet rows are also treated as visual annotation only and are not used as pipeline logic.

4 Experimental Pipeline

4.1 Evaluation Challenge

The project does not have ground-truth labels for director capacity, RFP effort, or correct director assignment. As a result, standard supervised-learning metrics such as accuracy, precision, recall, or F1-score are not appropriate as the primary evaluation method. A high numerical score would not be meaningful without a labeled definition of the correct assignment or an agreed business target.

Instead, the prototype is evaluated as a decision-support workflow. The experimental pipeline checks whether the system is implemented reproducibly, whether preprocessing preserves the expected data grain, whether retrieval returns inspectable evidence, whether the dashboard and RFP pages remain stable under realistic inputs, and whether the displayed outputs are appropriately caveated for stakeholder review.

4.2 Baselines and Enhanced Conditions

The baseline system behavior is deterministic and fallback-safe. For capacity scoring, the baseline is the formula-based capacity model built from cleaned CRM opportunity records. For RFP assignment, the baseline is deterministic local retrieval and heuristic recommendation logic that can

produce a dashboard-facing `assignment_context` without Azure embedding, Chroma, or LLM enrichment.

The enhanced system condition adds the preferred Azure OpenAI embedding plus Chroma retrieval path when configuration and dependencies are available. It also uses real or pre-built `capacity_df` in the RFP recommendation flow when available, optional Azure OpenAI chat enrichment for summary / effort / match-reason text, and alias-based CRM metadata linkage when deterministic `rfp_alias` matches are available.

4.3 Validation Layers

The experimental pipeline uses several validation layers:

- **Preprocessing and regression tests:** check that default and command-line workflows preserve expected data behavior, including configurable spreadsheet paths, configurable sheet names, missing file / missing sheet handling, and alias preservation.
- **Capacity-scoring checks:** verify that the capacity engine produces dashboard-ready fields such as `current_load`, `historical_avg_load`, `relative_load`, `capacity_score`, and `capacity_label`.
- **Retrieval validation:** check whether Azure configuration is correctly parsed and API embedding calls return valid vectors. We explicitly separate the validation into distinct layers: backend smoke tests, sample retrieval validation, and a full-corpus local proposal retrieval check. We also verify that local fallback retrieval triggers properly when Azure/Chroma is unavailable, while strictly refraining from claiming full browser-based app-flow validation unless manually verified.
- **Browser-style QA:** check that the Director Capacity Dashboard and RFP Assignment Tool load, display expected fields, handle empty or weak inputs, and label retrieval / capacity / LLM statuses clearly.
- **Evidence review:** inspect whether retrieved supporting examples actually support the displayed RFP outputs, especially across specific, mixed, and vague input types.

- **Stakeholder-facing wording review:** check that outputs are described as prototype decision support rather than final staffing decisions or proof of prior director experience.
- **Retrieval failure-mode assessment:** evaluate stakeholder awareness when Azure/Chroma is unavailable. This involves: (1) checking whether reviewers can consistently spot the active retrieval-mode indicator, and (2) adjusting their expectations accordingly when the system uses local heuristic fallback.

This section defines how the prototype is evaluated. The resulting observations and numerical validation outcomes are reported in Section 5.

5 Results and Analysis

5.1 Implementation and Regression Results

The final full-suite validation result was 168 passed, 2 warnings. The remaining warnings come from existing capacity-engine tests and do not indicate failures in the client data replacement workflow. This result improves on the previous automated baseline of 149 passed, 2 warnings. These tests provide implementation and regression confidence, but they should not be interpreted as business validation of director-capacity labels or final RFP assignment recommendations.

The data replacement workflow also preserved 19 non-empty RFP alias values in the cleaned output. This confirms that the newly provided machine-readable `Json S-Num` field is not dropped by the selected supplemental-field merge logic and can be used for deterministic RFP-to-CRM linkage review when a matching alias is available.

5.2 Capacity Dashboard Findings

The Director Capacity Dashboard launched successfully, loaded computed CRM-derived capacity data, and displayed director-level outputs including capacity labels, KPI cards, a relative-load chart, a label-mix chart, workload trends, and a director summary table. As shown in Figure 2, the page is organized around `relative_load`: stakeholders can immediately see which directors are below, near, or above their own historical workload norm, then inspect supporting workload drivers in the lower charts and table.

This display choice is more interpretable than centering the dashboard on a capped `capacity_score`. `relative_load` preserves the

Validation layer	What was checked	Observed result	Interpretation limit
Automated test suite	Full implementation and regression checks after handoff-oriented changes	168 passed, 2 warnings	Confirms implementation stability, not business correctness
Preprocessing replace-ment	CLI paths, configurable sheet name, default demo workflow, missing file / sheet behavior, alias preservation	Client data replacement path remained compatible with existing workflow	Still a local handoff workflow, not production ingestion
Alias preservation	Json S-Num preserved as rfp_alias / has_rfp_alias	19 non-empty alias values retained in cleaned output	Linkage provides context, not proof of director experience
Azure/Chroma backend retrieval	Azure configuration, embedding calls, Chroma vector queries, and fallback triggers	Preferred path was validated through backend smoke tests and local corpus queries	Refraining from full browser-side queries until those specific Streamlit paths undergo strict manual clicks and validations
Browser-style QA	Dashboard loading, RFP page display, retrieval labels, risk flags, empty/fallback states	Prototype remained demonstrable across selected scenarios	Scenario coverage remains limited

Table 3: Summary of major validation results and interpretation limits.

degree of overextension, while `capacity_score` becomes less informative once a director reaches or exceeds baseline. The dashboard results still require CGI review because workload is inferred from CRM opportunity signals rather than measured hours, calendar commitments, or manually confirmed availability.

5.3 RFP Retrieval and Assignment Findings

The RFP Assignment Tool accepted pasted sample RFP text and uploaded text-based inputs, retrieved supporting examples through the active retrieval path when configured, generated an RFP summary, displayed estimated effort, recommended directors using live or pre-built capacity data when available, displayed risk flags, and labelled retrieval and capacity-data status. Figure 3 shows the stakeholder-facing output structure: the page identifies the active retrieval mode, shows a prototype-output caveat, displays the submitted RFP input, lists Retrieved Supporting Examples with match scores and linked CRM context when available, presents capacity-aware recommended directors, and surfaces risk flags before the user treats the result as assignment guidance.

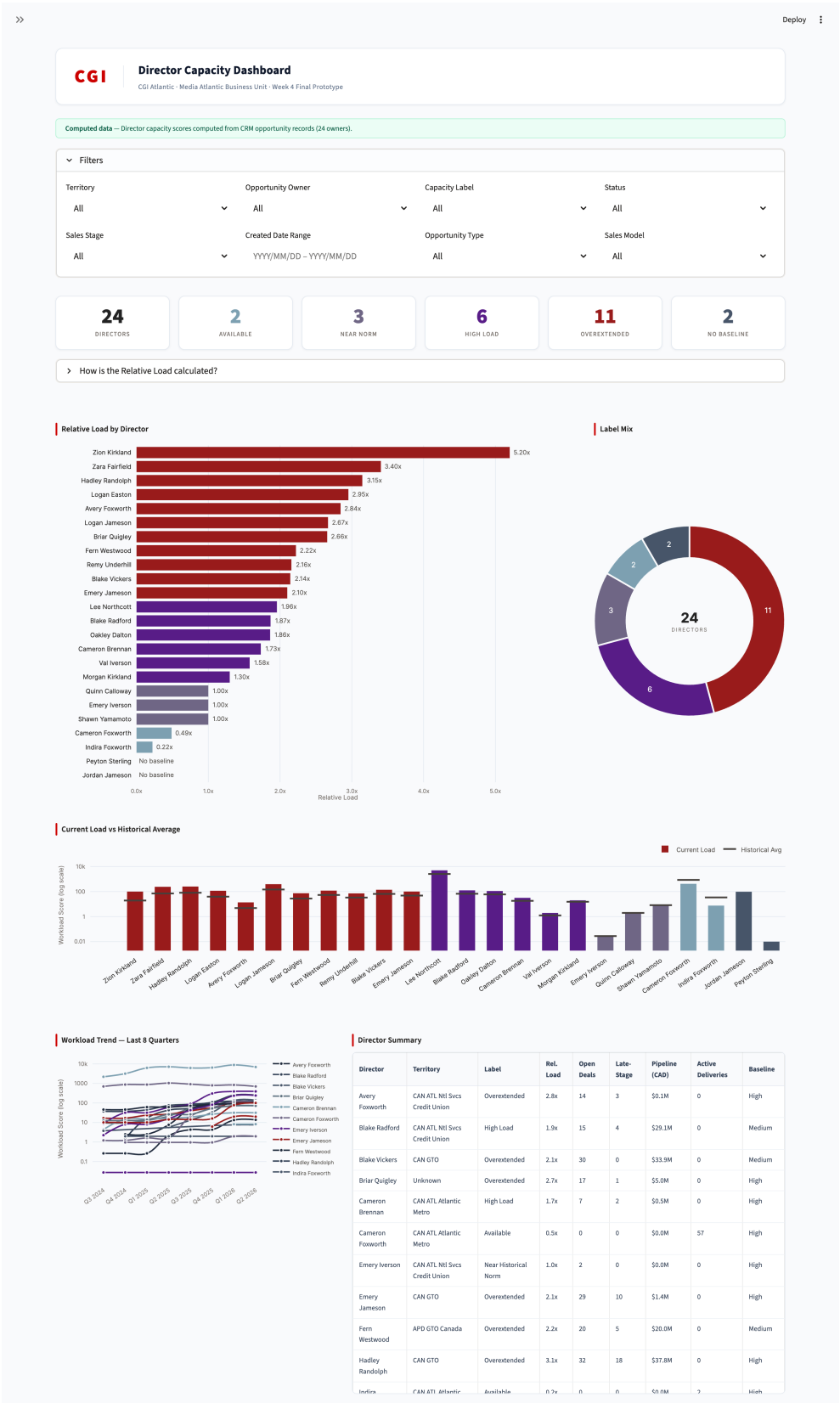
The evidence review tested three representative RFP input patterns: cybersecurity / compliance, cloud migration / managed services, and a vague short technology request. The cybersecurity / compliance input produced the strongest evidence because specific domain terms helped the system retrieve chunks related to compliance, risk, access controls, or security assessment work. The

cloud migration / managed services input produced mixed evidence: several retrieved chunks were thematically related to modernization or managed services, but some were broader than the submitted RFP and only weakly supported some displayed interpretations. The vague short input produced weak or broad evidence because the request did not provide enough scope detail for the retrieval system to distinguish a specific service need.

The review also checked whether retrieved examples supported displayed output fields, including summaries, match reasons, risk flags, and effort-related wording. A key finding is that retrieved chunks should not be described as direct proof for effort estimates or director experience. In the current prototype, effort logic is primarily heuristic and retrieval evidence provides contextual support rather than a validated duration label. This is why the output wording emphasizes retrieved supporting examples and risk flags instead of presenting the recommendation as a final staffing decision.

Taken together, these observations suggest a hypothesis: retrieval quality scales with input specificity. Specific RFPs give the system more precise service-domain signals, while vague requests increase the risk of broad semantic matches that look more confident than the evidence justifies. This hypothesis motivates stronger low-detail warnings and continued qualitative evidence review with CGI-provided examples.

Manual RFP workflow testing covered several input types that reflect likely stakeholder use cases. Cloud migration and managed-services inputs pro-



> How is the Relative Load calculated?

Relative Load by Director

Label Mix

Current Load vs Historical Average

Workload Trend — Last 8 Quarters

Director Summary

Director	Territory	Label	Rel. Load	Open Deals	Late-Stage	Pipeline (CAD)	Active Deliveries	Baseline
Avery Foxworth	CAN ATL Ntl Svcs Credit Union	Overextended	2.8x	14	3	\$0.1M	0	High
Blake Radford	CAN ATL Ntl Svcs Credit Union	High Load	1.9x	15	4	\$29.1M	0	Medium
Blake Vickers	CAN GTO	Overextended	2.1x	30	0	\$33.9M	0	Medium
Briar Quigley	Unknown	Overextended	2.7x	17	1	\$5.0M	0	High
Cameron Brennan	CAN ATL Atlantic Metro	High Load	1.7x	7	2	\$0.5M	0	High
Cameron Foxworth	CAN ATL Atlantic Metro	Available	0.5x	0	0	\$0.0M	57	High
Emery Iverson	CAN ATL Ntl Svcs Credit Union	Near Historical Norm	1.0x	2	0	\$0.0M	0	High
Emery Jameson	CAN GTO	Overextended	2.1x	29	10	\$1.4M	0	High
Fern Westwood	APD GTO Canada	Overextended	2.2x	20	5	\$20.0M	0	Medium
Hadley Randolph	CAN GTO	Overextended	3.1x	32	18	\$37.8M	0	High
Indira	CAN ATL Atlantic	Available	0.7x	0	0	\$0.0M	7	High

>>

Deploy

CGI

RFP Assignment Tool

CGI Atlantic • Media Atlantic Business Unit • Week 4 Final Prototype

Computed data — Director capacity scores computed from CRM opportunity records (24 owners).

Week 4 Final Prototype — RFP Assignment Tool

Retrieval: active mode is shown after analysis (Azure/Chroma when available, local fallback otherwise). Capacity data: real director capacity scores when available. Recommendations: prototype guidance only — not final CGI assignment decisions.

RFP Input

Upload RFP document

20-09aa-final.pdf

+

Loaded text from 20-09aa-rfp-for-case-management-software-final.pdf.

Upload a supported document or paste RFP text directly below.

schedule:

Activity Date

Pre-proposal Conference Call September 15, 2020 at 11:00am CST

Deadline to submit Questions via email to purchasing@duluthmn.gov

September 21, 2020

Answers to questions will be posted to the City website no later than

Territory

Service Domain

Any

Any

Territory and service domain are UI context only for now; backend scoring does not use them yet.

Analyze RFP

Retrieval mode — Azure/Chroma retrieval active.

Prototype output — capacity-aware recommendations using Computed from CRM opportunity records. Retrieval: Azure/Chroma retrieval active.

Azure/Chroma retrieval is active for this result.

Retrieval corpus: local proposals_responses.json corpus, 1216 corpus chunks, 11 submitted RFP query text; historical/sample corpus chunks.

Some retrieved examples are linked to CRM opportunities through CGI's RFP alias. Non-won linked opportunities are historical context, not proof of successful prior delivery. Linked opportunity owner reflects CRM ownership/context, not proof of personal delivery experience.

LLM enrichment: Azure OpenAI chat was used for the summary, effort estimate, and director match wording.

RFP Summary

The City of Duluth is seeking proposals for a case management software solution for its City Attorney's Office, including software, hardware, implementation, training, and ongoing support to replace the current system and improve operational efficiency.

Estimated Effort

High

Estimated duration: 12-16 weeks

The project involves multiple components such as software acquisition, hardware integration, implementation, training, and long-term support, requiring significant coordination and resources.

Retrieved Supporting Examples

1. 4_courts_case_management_system — match score 0.61

Proposal: 4_courts_case_management_system

Chunk ID: 4_courts_case_management_system_proposal_chunk_0031

Match score: 0.61

CRM opportunity: 18621656

Opportunity owner: Hadley Randolph

Outcome: open

= 100%. A Proponent who bids 150.00 receives 80% of the possible points for that category (126/150 = 88%), and a Proponent who bids 240.00 receives 50% of the possible points for that category (120/240 = 50%). Lowest rate ----- x Total available points = Score for second-lowest rate Second-lowest rate...

2. 3_data_sharing_platform_shared_analytics — match score 0.59

3. 4_courts_case_management_system — match score 0.59

Recommended Directors

1. Indra Foxworth

Capacity: Available

Capacity score: 0.78

Relative load: 0.22x

Assignment score: 0.68

Prototype fit is based on available capacity, service-fit signals, and retrieved semantic evidence; it does not prove prior director experience. LLM note: Indra's expertise in managing complex projects and her ability to oversee multi-phase implementations make her a strong candidate for this RFP.

Capacity explanation

Indra Foxworth is labelled Available with capacity score 0.78 and relative load 0.22. This is a heuristic input to the assignment score, not a final staffing decision.

Experience match explanation

Experience fit is estimated from retrieved historical/sample RFP chunks and simple service-keyword overlap (0.40). Retrieved examples include CRM opportunity linkage, but not to this director in the current top retrieved set.

Supporting chunks: 4_courts_case_management_system_proposal_chunk_0031, 3_data_sharing_platform_shared_analytics_proposal_chunk_0010

2. Cameron Foxworth

3. Jordan Jameson

Risk Flags

Low: Some retrieved examples are linked to CRM opportunities, but not all linked opportunities are won. Treat non-won links as historical context, not proof of successful prior delivery.

Medium: Some directors have no historical baseline, so capacity score and relative load use heuristic defaults in RFP ranking. Review No baseline candidates before assignment.

High: At least one director in the broader capacity pool is overextended based on capacity signals. Review workload before assigning outside the recommended shortlist.

Notes

Prototype output enriched with Azure OpenAI chat analysis. Retrieval used Azure/Chroma. Retrieved evidence searched the local proposals_responses.json corpus.

Figure 3: RFP Assignment Tool prototype. The page shows active retrieval mode, prototype-output caveats, RFP input, retrieved supporting examples, capacity-aware director recommendations, and risk flags.

Input type	Evidence quality	Interpretation
Cybersecurity compliance	Mostly relevant	Specific domain terms helped retrieval surface more useful supporting examples.
Cloud migration / managed services	Mixed	Retrieved chunks were often semantically related but sometimes broader than the submitted RFP.
Vague short technology request	Weak or broad	Short inputs can produce confident-looking service-fit signals without enough evidence.

Table 4: Qualitative retrieval patterns observed during evidence review. The table summarizes evidence quality, not assignment accuracy or business validation.

duced relevant modernization and operational-support examples, although some retrieved chunks were broader than the submitted RFP. Cybersecurity and compliance inputs produced some of the strongest retrieval results because the submitted text contained clearer domain-specific terms such as monitoring, incident response, vulnerability management, privacy, and audit preparation. Vague IT consulting inputs still returned summaries, effort estimates, retrieved examples, and director recommendations, but the evidence was weaker and more general. Procurement-only or deadline-focused inputs correctly demonstrated a limitation of the tool: the system can retrieve administratively similar RFP sections, but those examples should not be interpreted as reliable service-fit evidence.

The RFP page also handled operational workflow checks. Pasted text input worked, uploaded TXT files were parsed successfully, and empty input was handled without crashing. When Azure/Chroma retrieval was available, the page labelled the result as Azure/Chroma retrieval active. When Azure configuration was temporarily unavailable, the tool fell back to local retrieval and preserved the same output structure. In both modes, the page continued to return the expected fields: RFP summary, estimated effort, retrieved supporting examples, recommended directors, risk flags, and notes.

These tests suggest that the RFP Assignment Tool is useful as a first-pass review workflow. Its strongest use is not to make a final assignment

automatically, but to quickly summarize a new RFP, surface comparable historical examples, combine those examples with capacity information, and show stakeholders where the evidence is strong or weak.

5.4 Azure/Chroma and Fallback Behavior

Azure embedding access was validated at the backend level after CGI provided the remaining Azure OpenAI configuration values. The team ran smoke tests, generated embeddings, queried the Chroma vector store, and confirmed that the backend returned `retrieval_mode = azure_chroma` for local proposal-corpus checks. These checks included the full local proposal corpus of 1,216 chunks and confirmed that `generate_assignment_context()` can return `retrieval_mode` and `retrieval_status` metadata. These results support backend-level confidence in the preferred retrieval path, but they should not be interpreted as full production reliability or complete browser-flow validation.

The local fallback path remains necessary for robustness. The current retrieval code attempts to reuse an existing Chroma store when available and supports explicit rebuild behavior through configuration. If Chroma is unavailable, dependencies are missing, or query reuse fails, the system falls back to deterministic local retrieval while preserving the same dashboard-facing output schema. This design keeps the Streamlit demo path usable even when the preferred vector-store path cannot run.

5.5 Alias-Based CRM Linkage Analysis

The final prototype improved traceability by preserving the new `Json S-Num` field from the updated `opps1` spreadsheet as `rfp_alias / has_rfp_alias`. When proposal or RFP title `S-numbers` match cleaned opportunity aliases, retrieved examples can carry linked CRM context such as `opportunity_id`, `opportunity_owner`, `service_solution`, `status` fields, and `opportunity outcome`.

This is useful for reviewability, but it does not solve the broader evidence problem. Linked CRM metadata should not be interpreted as proof of proposal authorship, prior director involvement, delivery responsibility, staffing ownership, or final assignment suitability. If no deterministic alias match is available, retrieved examples remain semantic evidence only. Even when linked context is available, CGI review is still required before using

the output for staffing decisions.

5.6 Error Analysis and Main Takeaways

The most important error pattern is overconfidence under weak evidence. Short or underspecified RFP inputs can still generate summaries, match scores, service-fit signals, or recommendation text that looks polished. The system therefore needs to continue displaying low-confidence warnings and encouraging stakeholders to provide enough scope detail.

A second risk is evidence overinterpretation. Retrieved proposal or response chunks are useful as supporting semantic evidence, and linked CRM metadata improves reviewability when deterministic alias matches exist. However, neither semantic retrieval nor alias linkage proves that a director has relevant prior experience or should receive a new assignment.

A related limitation is that director recommendations can appear stable across different RFPs when the same directors have the strongest capacity signals. This is expected under the current scoring design because capacity is intentionally weighted heavily. The tool is meant to avoid recommending overloaded directors, so highly available directors will often remain near the top unless the RFP contains strong service-fit or linked CRM evidence that drastically changes the ranking. This behavior is the result of capacity-first recommendation logic, not as a bug in the retrieval system.

Alias linkage improves the situation by connecting some retrieved proposal examples to CRM opportunity records. However, the current linkage is still opportunity-level context, not a complete assignment history. A retrieved example may be tied to an opportunity owner or manager in the CRM data, but that does not necessarily confirm who authored the proposal, led the delivery, or should be assigned to a similar RFP in the future. For this reason, the safest interpretation is that the tool provides a reviewable shortlist supported by capacity and contextual evidence.

For example, in the cybersecurity / compliance review, the strongest results used domain-specific language such as access control, risk, compliance, or assessment, while weaker results tended to share only generic terms such as “assessment” or “modernization.” This pattern shows why match scores need to be read together with the retrieved text preview and risk flags.

The main project takeaway is that the prototype

is useful as a transparent handoff-ready decision-support workflow. A stakeholder can view director capacity results, inspect workload drivers, submit or paste RFP text, receive capacity-aware prototype recommendations when capacity data is available, review retrieved supporting examples, inspect linked CRM context when reliable alias matches exist, and see risk flags and caveats. The system is not yet a production-ready or business-validated assignment system.

6 Future Work

Future work should be framed as a staged roadmap rather than a list of unfinished tasks. The next stages should focus on CGI local testing, stakeholder review, handoff validation, and conservative refinement before major new feature expansion.

6.1 Immediate Next Steps: 1–2 Months

The immediate next step is CGI review of the current prototype. The capacity dashboard thresholds and label names should be reviewed with CGI to confirm whether they match business language. The RFP Assignment Tool should be evaluated with representative stakeholder examples, with particular attention to whether retrieved supporting examples are relevant, whether linked CRM context is understandable, whether risk flags are useful, whether vague-input warnings are strong enough, and whether recommendation wording remains appropriately caveated.

The team and CGI should also validate the new alias-based RFP-to-CRM linkage using real stakeholder knowledge. The current implementation preserves and propagates alias-linked metadata, but CGI needs to confirm whether the linkage is meaningful and how it should be presented in decision-support workflows.

Reviewers should also familiarize themselves with the tool’s retrieval failure modes. Whenever Azure or Chroma is unavailable, the system switches to local fallback while preserving the same dashboard-facing output structure. Stakeholders must make a habit of checking the retrieval-mode indicator on the dashboard so they know exactly which backend powered the current search. Catching this shift is crucial because the fallback engine relies on simpler heuristic matching rather than deep semantic embeddings, which naturally alters the quality of the surfaced evidence.

CGI should also review the recommendation

weighting with business stakeholders. In the current prototype, capacity is intentionally the strongest signal, while retrieval evidence, service-domain overlap, and linked CRM context provide supporting evidence. This weighting makes sense for a capacity-planning tool, but CGI may choose to adjust it depending on whether future use should prioritize availability, service expertise, relationship ownership, or historical win/loss outcomes.

6.2 Medium-Term Improvements: 3–6 Months

Medium-term work should calibrate recommendation weights and capacity thresholds with CGI feedback. The current assignment score combines capacity signals, retrieval evidence, keyword or service-domain overlap, and heuristic logic, but these weights have not been calibrated against stakeholder-approved outcomes or historical assignment labels.

Another useful medium-term improvement would be a larger and more complete proposal/RFP corpus. The current local proposal corpus is sufficient for prototype testing, but broader coverage would improve retrieval quality across more service domains and client contexts. More examples would also make it easier to evaluate whether retrieved chunks are consistently useful for stakeholder review.

Another improvement would be maintaining a persistent or prebuilt Chroma retrieval index. In the prototype, rebuilding or initializing the retrieval store can be slow when working with the full local corpus. A prebuilt index would let the app start faster, avoid repeated embedding work, and make the handoff workflow easier for CGI users who want to run the tool locally.

Retrieval validation should also be expanded across more representative RFPs. The current evidence review showed that specific inputs can produce useful retrieved examples, while vague or broad inputs can produce weak evidence. A larger validation set would help identify which domains, input lengths, and document types are most reliable.

Document upload parsing also remains prototype-level. DOCX and text-based PDF parsing should continue to be tested carefully, scanned PDFs may require manual paste or OCR outside the app, and large or malformed inputs require additional validation.

6.3 Longer-Term Production Readiness: 6–12 Months

Longer-term work should harden the Azure + Chroma retrieval path for production, including more robust index persistence, cache management, deployment-time dependency handling, and monitoring of retrieval behavior. First-time index creation, explicit rebuilds, and production-scale index management may require additional optimization.

If CGI wants to move beyond decision support toward stronger recommendation validation, the project would need stakeholder-approved labels or outcome data for capacity, RFP effort, and assignment quality. This would make it possible to evaluate model performance more directly and calibrate recommendations against business outcomes.

Finally, production use would require governance and operational planning: access controls, data refresh procedures, privacy controls for local spreadsheets and credentials, documentation ownership, and a clear process for reviewing or overriding recommendations.

7 Conclusion

This project set out to make director workload and RFP assignment more visible, consistent, and reviewable for CGI Atlantic / Media Atlantic. The final prototype, the CGI Capacity Analyzer, addresses this goal through two connected deliverables: a Director Capacity Dashboard and an RFP Assignment Tool. Together, these components help stakeholders inspect workload distribution, compare current director workload with historical norms, and review new RFP assignment suggestions with supporting evidence rather than relying only on informal or manual judgment.

The dashboard provides an interpretable view of director capacity by showing current workload, historical norm, relative workload, workload drivers, and capacity labels. The RFP Assignment Tool extends this workflow by combining capacity signals, retrieved supporting examples, service-fit evidence, and risk flags to suggest suitable directors for stakeholder review. The prototype also supports a more practical handoff workflow: CGI can update local data sources, regenerate processed outputs, and review CRM-linked context when reliable alias-based matches are available. Implementation tests, regression checks, retrieval validation, and app-level QA provide confidence that the prototype workflow is functional, but they do not replace busi-

ness validation.

The main limitation is that the system remains a decision-support prototype, not a production-ready staffing system. Retrieved supporting examples and linked CRM context should help stakeholders review a recommendation, but they should not be treated as proof of proposal authorship, prior director involvement, delivery responsibility, or final assignment suitability. The most important next step is CGI stakeholder validation: reviewing capacity labels, recommendation weights, retrieval evidence, alias linkage, risk warnings, and hand-off documentation against real assignment expectations.

Acknowledgements and Authorship Statement

We sincerely thank every team member for their effort and collaboration throughout this project. The team experienced membership changes, coordination challenges, technical implementation issues, and communication adjustments during the project. Even with these challenges, team members continued to complete their assigned work, review each other's contributions, and bring together the final report, prototype system, validation results, and handoff documentation.

We are also grateful to the UBC teaching team, especially our mentor Garrett, for guidance on project direction, report structure, technical framing, and stakeholder communication. We also thank CGI for providing a real-world project opportunity, project materials, and feedback throughout the capstone. This project helped us better understand the technical, collaborative, project-management, and professional communication skills required in applied data science work.

For authorship and AI-use transparency, the core problem framing, project goals, system design, technical decisions, experimental interpretation, result analysis, final content choices, and submission decisions were developed, reviewed, and approved by the human authors. ChatGPT was used only as an assistive tool for LaTeX organization, initial draft-ordering suggestions, content-organization suggestions, expression-level revision suggestions, wording polish suggestions, and checks against instructor feedback for structure and safe-claims language. All substantive revisions, technical judgments, acceptance or rejection of suggestions, final wording decisions, and final submission decisions

were made by the team members.

The writing and revision process is documented through the Time and Writing Management deliverable, the GitHub repository history, the project timeline, and the team contribution records. These records show the continuing process through which the report content, prototype implementation, and documentation were developed, reviewed, and refined by the team.

A Timeline and Team Contributions

This appendix summarizes team contributions and preserves the project timeline in a compact form. The main report describes the project motivation, data, methods, experimental pipeline, results, and future work. This appendix focuses on who contributed to the main workstreams and how the prototype developed across phases.

A.1 Team Contributions by Member

Yixiao. Led the overall report structure, safe-claims framing, data integration, and final handoff alignment. Early work focused on merging the two anonymized CRM opportunity exports, preserving opportunity-level grain, comparing schemas, handling duplicate and unmatched records, and producing downstream-ready opportunity data for validation, scoring, and dashboard development. Later work focused on coordinating RFP integration, stabilizing the `assignment_context` contract, tracking QA evidence, documenting reproducibility and limitations, and ensuring that the report did not overclaim business validation. In the final handoff stage, Yixiao implemented configurable local spreadsheet replacement for CGI-provided opportunity files, preserved the RFP alias field as `rfp_alias` / `has_rfp_alias`, supported local processed-output regeneration, and helped align final claims around prototype readiness rather than production staffing decisions.

Freya. Led the stakeholder-facing dashboard and RFP UI work across the project. Early work moved the Streamlit pages from skeleton screens toward an integrated capacity dashboard and an RFP page that could display `assignment_context` outputs. Later work refined the relative-load-first dashboard interpretation, clarified filter semantics and capacity-label wording, connected capacity data into the RFP display path, and polished the output structure for summaries, effort estimates, retrieved examples, recommended directors, risk flags, and notes. Freya also improved stakeholder-facing wording and terminology by using labels such as Retrieved Supporting Examples, separating retrieval and LLM enrichment statuses, rounding director metrics, clarifying capacity-data-source captions, and maintaining conservative director-experience caveats. In the final stage, Freya implemented alias-based RFP-to-CRM linkage in the RFP pipeline so retrieved examples can surface linked CRM context when de-

terministic `rfp_alias` matches are available, while preserving the interpretation that linked metadata supports review rather than proving proposal authorship, director experience, or final assignment suitability.

Kian. Led data validation, evidence review, and conservative interpretation of retrieved examples. Early work validated the fields needed for capacity scoring and dashboard use, including revenue, date, duration, probability, status, sales stage, opportunity owner, and opportunity manager fields. Kian also validated the proposal JSON structure, extracted proposal and response text, checked chunk metadata, and documented the limitation that proposal chunks alone cannot reliably prove director or opportunity ownership. Later work focused on retrieval-evidence review across representative RFP inputs, including cloud / managed services, cybersecurity / compliance, and vague short requests. This review helped establish that retrieval evidence quality depends strongly on input specificity and that retrieved examples should support review, not be treated as direct proof of effort, director experience, or assignment correctness. Kian also contributed regression tests for the client data replacement workflow and helped keep evidence-related wording conservative.

Lyken. Led the capacity scoring backend and Azure/Chroma retrieval validation work. Early work translated the capacity-scoring skeleton into a functional owner-level scoring workflow, producing director-level capacity outputs such as relative load, capacity score, capacity labels, and scoring sanity checks. Lyken also advanced the retrieval backend from local fallback behavior toward Azure OpenAI embeddings and Chroma vector-store retrieval while preserving deterministic fallback retrieval. Later work focused on configuration checks, backend smoke tests, metadata routing, local proposal-corpus retrieval checks, and confirmation that retrieval metadata such as `retrieval_mode` and `retrieval_status` could be passed to the RFP output. In the final stage, Lyken checked that the Streamlit dashboard and RFP page still loaded after regenerated opportunity data was introduced, supporting the claim that the handoff-oriented changes did not break the main app workflow.

Jai. Led RFP preprocessing, recommendation logic, QA scenarios, and handoff troubleshooting. Early work created the RFP preprocessing skeleton, chunk generation logic, mock or basic re-

trieval output, and a stable assignment_context schema for dashboard integration. Later work improved generate_assignment_context() so the RFP tool could combine retrieved examples, available capacity fields, service-fit signals, risk flags, and heuristic recommendation logic. Jai also tested the RFP page across varied inputs, including cloud/data modernization, cybersecurity, vague IT consulting, procurement/deadline-only text, large multi-domain transformation RFPs, empty input, TXT upload, and fallback mode. These tests helped document remaining caveats around broad or vague input, first-time retrieval-index creation, fallback behavior, and proposal-to-CRM linkage assumptions. In the final handoff stage, Jai reviewed the workflow from a CGI user perspective and documented common troubleshooting cases such as local spreadsheet placement, preprocessing reruns, missing processed outputs, missing alias columns, privacy warnings, and fallback behavior.

A.2 Phase-by-Phase Timeline

The following tables preserve the phase-by-phase implementation record in a compact form. They summarize each workstream's owner, focus, and output without repeating the longer narrative already covered in the main report.

Track	Owner	Phase 1 focus	Main output
Data foundation	Yixiao	Built the first reproducible opportunity-data merge and checked schema overlap, duplicate IDs, and unmatched records.	Opportunity-level merge pipeline; audit summary showing 8,746 final rows and zero duplicate opportunity_id values.
Data validation	Kian	Reviewed revenue, date, duration, probability, status, and sales-stage fields for downstream capacity use.	Initial validation notes for reliable, risky, and fallback-dependent fields.
Capacity scoring	Lyken	Created the initial capacity-scoring skeleton and director table prototype.	Early capacity logic and director-level output structure.
Dashboard	Freya	Created the Streamlit dashboard skeleton and initial page layout.	Dashboard placeholder with planned capacity display components.
RFP pipeline	Jai	Created the RFP preprocessing and vector-store skeleton.	Initial files for RFP text processing, chunking, and retrieval-oriented development.

Table 5: Phase 1 progress toward the data foundation and module skeletons.

Module	Owner	Phase 2 focus	Main output
Data integration	Yixiao	Refined the Phase 1 merge output into a downstream-ready cleaned dataset.	cleaned_opportunity_df, column dictionary, quality flags, and owner-level summary.
Data validation	Kian	Validated fields needed for capacity scoring and dashboard use.	Missingness summary, reliable / risky fields, revenue fallback hierarchy, and date / duration assumptions.
Capacity scoring	Lyken	Implemented the first working owner-level capacity scoring workflow.	director_capacity_df, relative load, capacity score, capacity label, and scoring sanity checks.
Dashboard	Freya	Connected the dashboard to real or schema-aligned scoring output.	Capacity score chart, owner-level table, filters, and explanation placeholders.
RFP pipeline	Jai	Built an independently runnable RFP preprocessing and retrieval-style prototype.	Sample chunks, mock or basic retrieval output, and assignment_context schema.
Consistency pass	Yixiao, Freya	Aligned architecture, capacity scoring, dashboard behavior, RFP retrieval, and output contracts after the latest RFP/capacity merge.	Fixed scoring gates, active-quarter baseline/trend logic, dashboard live-data path, stale RFP retrieval, and related docs/tests.

Table 6: Phase 2 progress from separate skeletons toward the first integrated prototype.

Track	Owner	Phase 3 focus	Main output
Capacity dashboard refinement	Freya	Implemented and documented the <code>relative_load</code> -first dashboard interpretation and updated capacity-label semantics.	Updated dashboard charts, filter semantics, workload formulas, documentation, and tests.
RFP integration / QA	Yixiao	Coordinated RFP pipeline status, API / fallback behavior, assignment schema, smoke testing, and documentation.	API usage notes, finalized <code>assignment_context</code> schema, smoke test checklist, integration QA notes, and risks.
RFP data validation	Kian	Validated <code>proposals_responses.json</code> , text extraction quality, chunk metadata, and proposal-to-director linkage limitations.	JSON structure summary, chunk metadata dictionary, sample chunks, and RFP data validation notes.
Azure embedding / vector store	Lyken	Advanced the code path from local fallback toward Azure embeddings and Chroma while preserving fallback retrieval; real Azure smoke testing remained blocked by missing API version and deployment configuration.	Azure client helper updates, vector store structure, fallback retrieval behavior, and documented blockers for Azure / Chroma completion.
RFP dashboard integration	Freya	Moved the Streamlit RFP page from placeholder toward displaying <code>assignment_context</code> .	RFP input UI, result display, retrieved examples, recommended directors, risk flags, and display notes.
RFP assignment logic	Jai	Improved <code>generate_assignment_context()</code> toward heuristic recommendation output using retrieval examples, available capacity fields when supplied, and risk flags.	Stable assignment output, heuristic effort logic, recommendation notes, risk flag logic, and test updates.

Table 7: Phase 3 progress toward RFP Assignment Tool integration.

Track	Owner	Phase 4 focus	Main output
RFP contract consistency pass	Yixiao, Freya	Tightened the RFP assignment contract before final integration.	Stable assignment_context; effort.reason primary and rationale alias; safer fallback chunks; non-finite numeric cleanup; regression tests; 108 passed.
Final integration QA	Yixiao	Froze final prototype scope, demo path, safe claims, limitations, and reproducibility notes.	QA checklist, demo checklist, architecture-compliance notes, final caveats, and report-ready scope language.
RFP data validation	Kian	Validated proposal structure, chunk metadata, retrieval examples, and linkage limits.	Final validation summary, representative retrieval examples, chunk metadata notes, and proposal-to-director / opportunity linkage caveats.
Azure / Chroma retrieval	Lyken	Integrated Azure OpenAI embeddings and Chroma retrieval into the active RFP flow, with fallback preserved.	Azure environment check, embedding validation, Chroma retrieval path, retrieval-mode notes, and fallback behavior.
RFP output quality and LLM enrichment	Freya	Polished RFP UI, wired real capacity data, and added Azure chat enrichment.	Upload/paste labels, severity risk flags, capacity_df handoff, LLM-enriched summary / effort / match reasons, and fallback-safe tests/docs.
RFP page stability	Jai	Verified page display and stability across normal, fallback, missing-field, empty, and error states.	Assignment-context display checks, sample RFP tests, retrieval-mode display, and stability notes.
Final prototype validation	Freya	Finalized the architecture-complete prototype.	Improved upload, retrieval, service-fit, LLM enrichment, and fallback behavior; documented in final_prototype_validation.md.

Table 8: Phase 4 progress toward final prototype stabilization and architecture-complete RFP assignment flow.

Track	Owner	Phase 5 focus	Main output
Final app QA and scope tracking	Yixiao	Coordinated user-style QA, issue/fix/decision tracking, teammate follow-up, and final claim-boundary alignment.	Final QA tracker, automated baseline of 149 passed, 2 warnings, manual dashboard and RFP checks, scope/caveat notes, Chroma reuse and fallback-safe retrieval stability fix, and report-ready safe wording.
Evidence and output review	Kian	Reviewed whether retrieved examples support displayed RFP outputs across cloud, cybersecurity, and vague short input cases.	Evidence review notes, effort-support checks, weak-evidence cases, and semantic-evidence boundary language.
Retrieval and config validation	Lyken	Validated Azure configuration, Azure embedding, Azure/Chroma retrieval, retrieval metadata, and fallback behavior.	Backend smoke tests and local proposal-corpus retrieval checks; retrieval_mode / retrieval_status reporting; fallback-safe retrieval behavior.
Dashboard usability and wording	Freya	Reviewed stakeholder-facing dashboard and RFP output clarity, including retrieval, fallback, LLM, and linkage caveat wording.	Retrieved Supporting Examples label, match-score wording, separate LLM enrichment status, rounded director metrics, and safer director-experience caveats.
Assignment logic review	Jai	Tested recommendation behavior, risk flags, fallback mode, upload behavior, and proposal-to-CRM linkage assumptions.	Varied RFP test notes, softened match-reason wording, fallback test result, remaining broad/vague-input caveats, and proposal-to-CRM crosswalk limitation notes.

Table 9: Phase 5 progress from architecture-complete prototype development toward user-style testing, evidence review, final wording refinement, and scope/caveat consolidation.

Track	Owner	Phase 6 focus	Main output
Client data replacement pipeline	Yixiao	Added command-line spreadsheet inputs for client-provided opportunity files while preserving the original no-argument demo workflow.	Configurable opps1 / opps2 paths, sheet name, and output directory; local processed-output regeneration under data/processed; preserved rfp_alias / has_rfp_alias; 19 non-empty alias rows retained in cleaned output.
RFP-to-CRM linkage	Freya	Used CGI's machine-readable RFP alias field to connect historical proposal/RFP records to CRM opportunity rows when deterministic alias matches are available.	Alias-based linkage from proposal title S-numbers to rfp_alias; linked CRM metadata including opportunity_id, opportunity_owner, service_solution, status fields, and opportunity outcome; conservative UI/docs wording that treats linkage as context rather than proof.
Regression and evidence review	Kian	Checked that client data replacement does not break existing preprocessing and that evidence wording remains conservative.	Tests for default and CLI workflows, configurable sheet names, missing file / missing sheet behavior, and alias preservation; evidence-review support for weak, unclear, non-won, or semantic-only retrieved examples.
App loading and processed-data QA	Lyken	Validated that the Streamlit app still works after opportunity data is regenerated through the new preprocessing workflow.	Processed-data loading checks; Director Capacity Dashboard smoke test; RFP page post-replacement smoke test; confirmation that dashboard metrics and retrieval ranking logic were not changed.
Handoff troubleshooting and guide review	Jai	Reviewed the workflow from a CGI user perspective and documented common failure cases.	Client-guide and troubleshooting notes covering local spreadsheet placement, preprocessing reruns, missing processed outputs, missing alias columns, privacy / gitignore warnings, and conservative caveats.
Full-suite validation	Team	Confirmed that the updated handoff-oriented workflow remains compatible with the existing codebase.	Latest full test suite result: 168 passed, 2 warnings; no raw CGI spreadsheet or generated processed output committed.

Table 10: Phase 6 progress from final prototype refinement toward client data replacement and handoff readiness.

References

- Chroma. 2026. Chroma Documentation. <https://docs.trychroma.com/docs/overview/introduction>. Accessed 23 June 2026.
- Microsoft. 2026a. Azure OpenAI in Microsoft Foundry Models: Embeddings Tutorial. <https://learn.microsoft.com/en-us/azure/foundry/openai/tutorials/embeddings>. Accessed 23 June 2026.
- Microsoft. 2026b. Azure OpenAI in Microsoft Foundry Models REST API Reference. <https://learn.microsoft.com/en-us/azure/foundry/openai/reference>. Accessed 23 June 2026.
- Snowflake Inc. 2026. Streamlit Documentation. <https://docs.streamlit.io/>. Accessed 23 June 2026.